This tutorial refers to a product that has reached its end-of-life status.

Controlling RGB LED Through Bluetooth®

Learn how to control the built in RGB LED on the Nano 33 BLE board over Bluetooth®, using an app on your phone.

Author · Fabricio Troya Last revision · 07/17/2024

In this tutorial we will use an Arduino Nano 33 BLE, to turn on an RGB LED over Bluetooth®, made possible by the communications chipset embedded on the board.

Goals

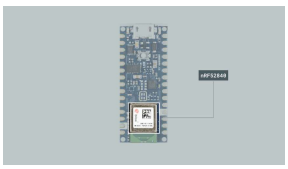
The goals of this project are:

- Learn what Bluetooth® Low Energy and Bluetooth® are.
- Use the Arduino BLE library.
- Learn how to create a new service.
- Learn how to turn on a RGB LED from an external device (smartphone).

Hardware & Software Needed

- This project uses no external sensors or components.
- In this tutorial we will use the **Arduino Cloud Editor** to program the board.

Bluetooth® Low Energy and Bluetooth®



The nRF52840 module.

Bluetooth® Low Energy, referred to as BLE, separates itself from what is now known as "Bluetooth® Classic" by being optimized to use low power with low data rates. There are two different types of Bluetooth® devices: central or peripheral. A central Bluetooth® device is designed to read data from peripheral devices, while the peripheral devices are designed to do the opposite. Peripheral devices continuously post data for other devices to read, and it is precisely what we will be focusing on this tutorial.

Service & Characteristics

A service can be made up of different data measurements. For example, if we have a device that measures wind speed, temperature and humidity, we can set up a service that is called "Weather Data". Let's say the device also records battery levels and energy consumption, we can set up a service that is called "Energy information". These services can then be subscribed to central Bluetooth® devices.

Characteristics are components of the service we mentioned above. For example, the temperature or battery level are both characteristics, which record data and update continuously.

Unique Universal Identifier (UUID)

When we read data from a service, it is important to know what type of data we are reading. For this, we use UUIDs, who basically give a name to the characteristics. For example, if we are recording temperature, we want to label that characteristic as temperature, and to do that, we have to find the UUID, which in this case is "2A6E". When we are connecting to the device, this service will then appear as "temperature". This is very useful when tracking multiple values.

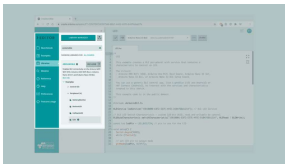
If you want to read more about UUIDs, services, and characteristics, check the links below:

- GATT services.
- GATT characteristics.

Creating the Program

1. Setting up

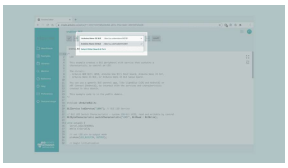
Let's start by opening the Arduino Cloud Editor, click on the **Libraries** tab and search for the **ArduinoBLE** library. Then in **> Examples > Peripheral**, open the **LED** sketch and once it opens, you could rename it as desired.



Finding the library in the Cloud Editor.

2. Connecting the board

Now, connect the Arduino Nano 33 BLE to the computer and make sure that the Cloud Editor recognizes it, if so, the board and port should appear as shown in the image below. If they don't appear, follow the instructions to install the plugin that will allow the Editor to recognize your board.



Selecting the board.

3. Turning ON the LED

Now we will need to modify the code on the example, in order to turn the RGB LED on in different colors, based on the information we sent through the smartphone.

After including the `ArduinoBLE.h` library, in the `ledService()` function change the argument to `"180A"` which is translated to **"Device Information"**. Then in the `switchCharacteristic()` function change the argument to `"2A57"` which is translated to **"Digital Output"**.

```
BLEService ledService("

// BLE LED Switch Chara
BLEByteCharacteristic s
```

In the `setup()` after initializing the serial communication we set the LED's pins as `OUTPUT`, and then turn them OFF, including the `LED_BUILTIN` that we will use to show if we are connected to the board.

```
// set LED pin to outp
pinMode(LEDR, OUTPUT);
pinMode(LEDG, OUTPUT);
pinMode(LEDB, OUTPUT);
pinMode(LED_BUILTIN, O

digitalWrite(LED_BUILT
digitalWrite(LEDR, HIG
digitalWrite(LEDG, HIG
digitalWrite(LEDB, HIG
```

In the `BLE.setLocalName()` we need to change the name to `"Nano 33 BLE"` in order to identify the board we are using.

```
// set advertised local
  BLE.setLocalName("Nan
  BLE.setAdvertisedServ
```

Now, in the `loop()` after the `Serial.println(central.addre ss());` we need to set the `LED_BUILTIN` to `HIGH` to turn on the LED when the phone is connected to the

```
// print the central's
Serial.println(central.
digitalWrite(LED_BUILTI
```

Then we replace the second `if()` function inside the `while` loop with a `switch case` function, where each case will turn each color of the LED.

```
// while the central
while (central.connec

// if the remote devi
// use the value to c

if (switchCharacteris
switch (switchCharact
  case 01:
    Serial.println("R
    digitalWrite(LEDR
    digitalWrite(LEDG
    digitalWrite(LEDB
    break;
  case 02:
    Serial.println("G
    digitalWrite(LEDR
    digitalWrite(LEDG
    digitalWrite(LEDB
    break;
  case 03:
    Serial.println("B
    digitalWrite(LEDR
    digitalWrite(LEDG
    digitalWrite(LEDB
    break;
  default:
    Serial.println(F(
```

Lastly, after the last `Serial.println()` function we use the `digitalWrite()` to turn off the LED once the phone is disconnected from the board.

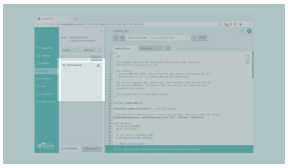
```
// when the central dis
Serial.print(F("Disconn
Serial.println(central.
digitalWrite(LED_BUILTI
digitalWrite(LEDR, HIGH
digitalWrite(LEDG, HIGH
digitalWrite(LEDB, HIGH
}
}
```

4. Complete code

If you choose to skip the code building section, the complete code can be found below:

```
#include <ArduinoBLE

BLEService ledServic

// BLE LED Switch Ch
BLEByteCharacteristi

void setup() {
  Serial.begin(9600)
  while (!Serial);

  // set LED's pin t
  pinMode(LEDR, OUTP
  pinMode(LEDG, OUTP
  pinMode(LEDB, OUTP
  pinMode(LED_BUILTI

  digitalWrite(LED_B
  digitalWrite(LEDR,
  digitalWrite(LEDG,
  digitalWrite(LEDB,

  // begin initializ
  if (!BLE.begin())
    Serial.println("

    while (1);
  }
```

Once we are finished with the coding, we can upload the sketch to the board. When it has successfully uploaded, open the Serial Monitor. In the Serial Monitor, the text **"BLE LED Peripheral"** will appear as seen in the image below.



Serial Monitor output.

We can now discover our Nano 33 BLE board in the list of available Bluetooth® devices. To access the service and characteristic we recommend using the **LightBlue** application. Follow this link for iPhones or this link for Android phones.

Once we have the application open, follow the image below for instructions:

Accessing through a Bluetooth® phone app.

To control the RGB LED, we simply need to write 1,2 or 3 in the "WRITTEN VALUES" field to turn on the red, blue or the green LED and any other value to turn them off. This is within the **"Digital Output"** characteristic, which is located under **"Device**

Troubleshoot

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

- Missing a bracket or a semicolon.
- Arduino board connected to the wrong port.
- We haven't opened the Serial Monitor to initialize the program.
- The device you are using to connect has its Bluetooth® turned off.

Conclusion

In this tutorial we have created a basic Bluetooth® peripheral device. We learned how to create services and characteristics, and how to use UUIDs from the official Bluetooth® documentation. Lastly, we turn on different colors of the RGB LED based on the values sent from the smartphone.

Now that you have learned a little bit how to use the ArduinoBLE library, you can try out some of our other tutorials for the Nano 33 BLE board. You can also check out the **ArduinoBLE** library for more examples and inspiration for creating Bluetooth® projects!

Was this article helpful?